

 PATRONES DE DESARROLLO DE SOFTWARE	Departamento: Ciencias de la Computación
	Carrera: Ingeniería en Software Asignatura: Análisis y diseño de Software NRC: 30723
	Apellidos y nombres del estudiante: Diana Guerra, Paul Moreno, Leonel Tipan

ANTECEDENTES

1. El documento base de la unidad plantea una progresión lógica para el desarrollo de software, comenzando por la arquitectura de tres capas y avanzando hacia la aplicación de patrones de diseño estructurales y comportamentales
2. Se utiliza un caso de estudio integrado enfocado en el CRUD de un estudiante (con atributos mínimos como ID, Nombres y Edad) para que funcione como hilo conductor y permita aplicar los conceptos en un contexto verificable, documentado y trazable a partir de la Especificación de Requisitos de Software (ERS/SRS)

OBJETIVO

1. Diseñar e implementar una arquitectura de tres capas (Datos, Lógica de Negocio y Presentación) que separe claramente las responsabilidades del sistema
2. Aplicar patrones de diseño para resolver problemas específicos: Adapter y Decorator para mejorar la estructura (adaptación de formatos externos y extensión de comportamiento), y Observer y Strategy para gestionar notificaciones y seleccionar comportamientos de forma dinámica

DESARROLLO

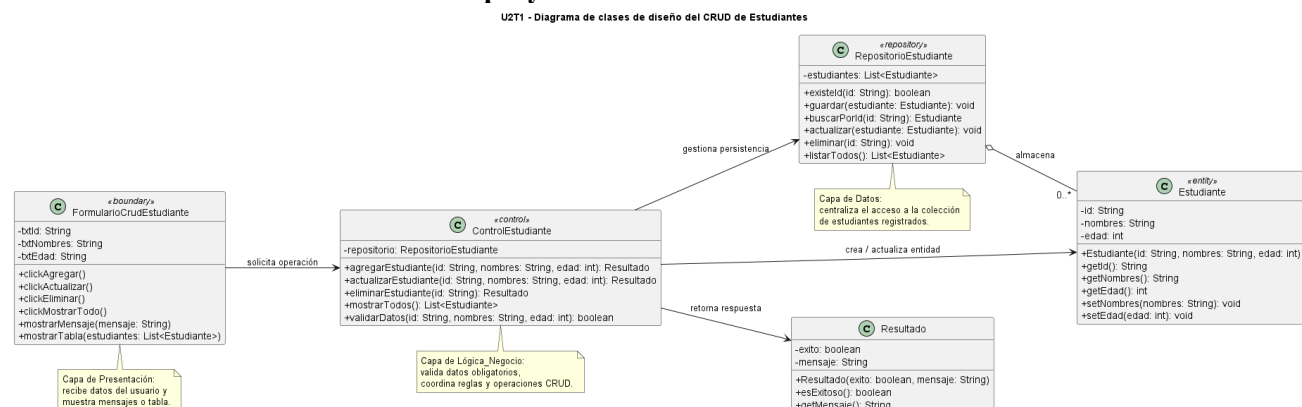
U2T1. Diseño e implementación de la arquitectura de tres capas para el CRUD del estudiante

1. Análisis de la ERS/SRS y selección de requisitos funcionales

De acuerdo con la Especificación de Requisitos, el sistema debe satisfacer las siguientes necesidades para el CRUD del estudiante:

- **RF-01:** Agregar un nuevo estudiante con ID, Nombre y Edad.
- **RF-02:** Actualizar los datos de un estudiante existente usando ID, Nombre y Edad.
- **RF-03:** Eliminar un estudiante seleccionado.
- **RF-04:** Mostrar todos los estudiantes registrados en una tabla.
- **RF-05:** Validar los datos obligatorios antes de guardar o actualizar cambios.

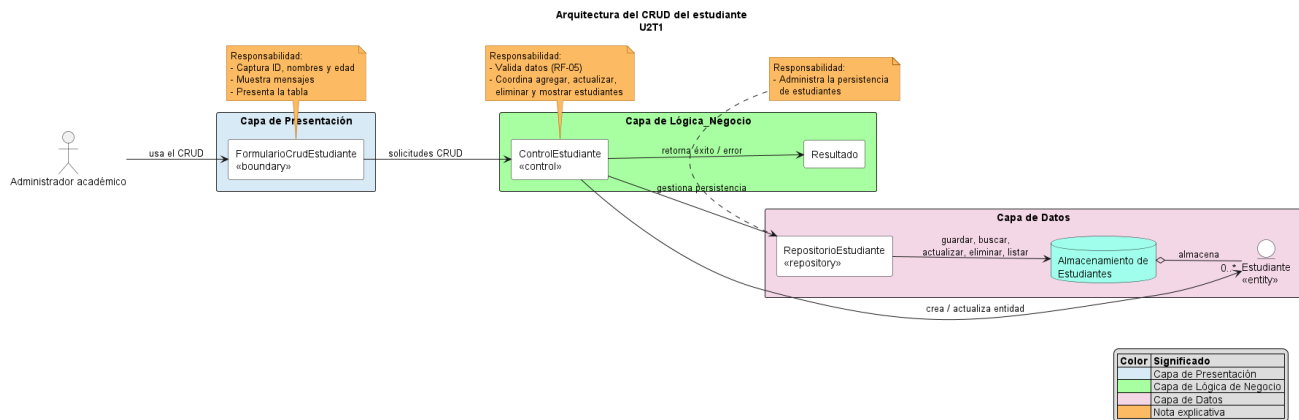
2. Identificación de las clases base del proyecto



A partir del diagrama de clases de diseño, las clases base que conforman el proyecto son:

- **Estudiante <<entity>>**: Entidad del dominio que contiene los atributos -id: String, -nombres: String y -edad: int, junto con sus métodos getters y setters.
- **FormularioCrudEstudiante <<boundary>>**: Clase de la interfaz con los campos de texto (txtId, txtNombres, txtEdad) y métodos para capturar eventos de los botones.
- **ControlEstudiante <<control>>**: Servicio principal que contiene la lógica para agregar, actualizar, eliminar, mostrar y validar datos. Retorna objetos de tipo Resultado.
- **RepositorioEstudiante <<repository>>**: Clase de persistencia que maneja la colección en memoria (-estudiantes: List<Estudiante>) y expone los métodos CRUD a nivel de datos.
- **Resultado**: Clase auxiliar utilizada por el control para retornar respuestas con atributos de éxito (-exito: boolean) y mensajes informativos (-mensaje: String).

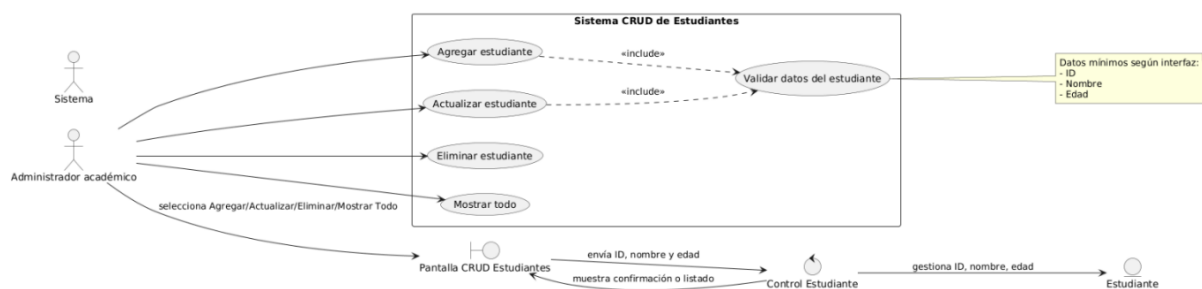
3. Diseño de la arquitectura en tres capas



El sistema divide estrictamente sus operaciones para garantizar que el Administrador académico interactúe únicamente con la vista, aislándolo de los datos:

- **Capa de Presentación (color azul):** Implementada por **FormularioCrudEstudiante**. Tiene la responsabilidad exclusiva de capturar el ID, nombres y edad ingresados por el usuario, enviar las solicitudes CRUD a la capa lógica, mostrar mensajes de éxito/error y presentar la tabla de datos.
- **Capa de Lógica_Negocio (color verde):** Implementada por **ControlEstudiante**. Actúa como orquestador. Su responsabilidad principal es validar los datos para cumplir con el RF-05 y coordinar las acciones de agregar, actualizar, eliminar y mostrar. Transforma las "solicitudes CRUD" de la vista en acciones de persistencia y retorna objetos de tipo Resultado (éxito/error) hacia la presentación.
- **Capa de Datos (color rosa):** Implementada por **RepositorioEstudiante** y el "Almacenamiento de Estudiantes". Su responsabilidad es administrar la persistencia. Recibe instrucciones exclusivas de la capa de Lógica para guardar, buscar, actualizar, eliminar y listar, y es la única capa que almacena instancias de la entidad Estudiante.

4. Implementación del CRUD base del estudiante



5. Responsabilidad de cada capa y relación

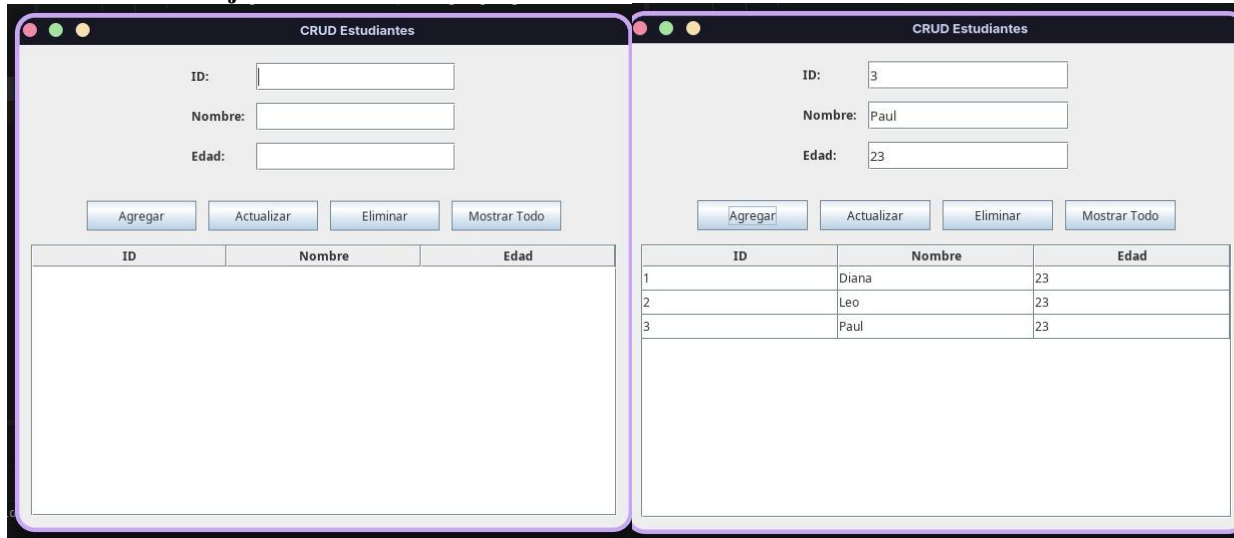
Trazabilidad:

RF	Requisito funcional	Caso de uso asociado	Clases de análisis relacionadas	Secuencia de análisis
RF-01	El sistema debe permitir agregar un estudiante con ID, Nombre y Edad.	Agregar estudiante	FormularioCrudEstudiante <<boundary>>ControlEstudiante <<control>>Estudiante <<entity>>RepositorioEstudiante <<repository>>	Secuencia: Agregar estudiante
RF-02	El sistema debe permitir actualizar los datos de un estudiante existente.	Actualizar estudiante	FormularioCrudEstudiante <<boundary>>ControlEstudiante <<control>>Estudiante <<entity>>RepositorioEstudiante <<repository>>	Secuencia: Actualizar estudiante
RF-03	El sistema debe permitir eliminar un estudiante existente cuando corresponda.	Eliminar estudiante	FormularioCrudEstudiante <<boundary>>ControlEstudiante <<control>>Estudiante <<entity>>RepositorioEstudiante <<repository>>	Secuencia: Eliminar estudiante
RF-04	El sistema debe permitir mostrar todos los estudiantes registrados.	Mostrar todos los estudiantes	FormularioCrudEstudiante <<boundary>>ControlEstudiante <<control>>Estudiante <<entity>>RepositorioEstudiante <<repository>>	Secuencia: Mostrar todos los estudiantes
RF-05	El sistema debe validar los datos obligatorios antes de agregar o actualizar.	Validar datos del estudiante	FormularioCrudEstudiante <<boundary>>ControlEstudiante <<control>>Estudiante <<entity>>	Se integra dentro de las secuencias: Agregar estudiante y Actualizar estudiante

Responsabilidad de cada capa

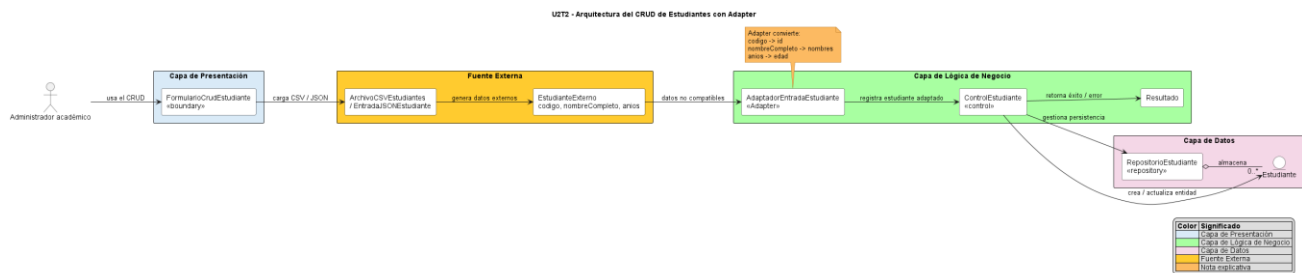
Requisito Funcional	Capa de Presentación (FormularioCrudEstudiante)	Capa Lógica de Negocio (ControlEstudiante)	Capa de Datos (RepositorioEstudiante)
RF-01 (Agregar)	+clickAgregar ()	+agregarEstudiante (id, nombres, edad) +validarDatos(...)	+existId(id)+guardar(estudiante)
RF-02 (Actualizar)	+clickActualizar ()	+actualizarEstudiante (id, nombres, edad) +validarDatos(...)	+buscarPorId(id)+actualizar(estudiante)
RF-03 (Eliminar)	+clickEliminar ()	+eliminarEstudiante(id)	+eliminar(id)
RF-04 (Mostrar todo)	+clickMostrarTodo ()	+mostrarTodos ()	+listarTodos ()
RF-05 (Validar datos)	Mostrar resultado mediante +mostrarMensaje(mensaje)	+validarDatos (id, nombres, edad) evaluando antes de procesar y retornando clase Resultado.	N/A (Regla de negocio aislada de los datos)

6. Evidencias de ejecución



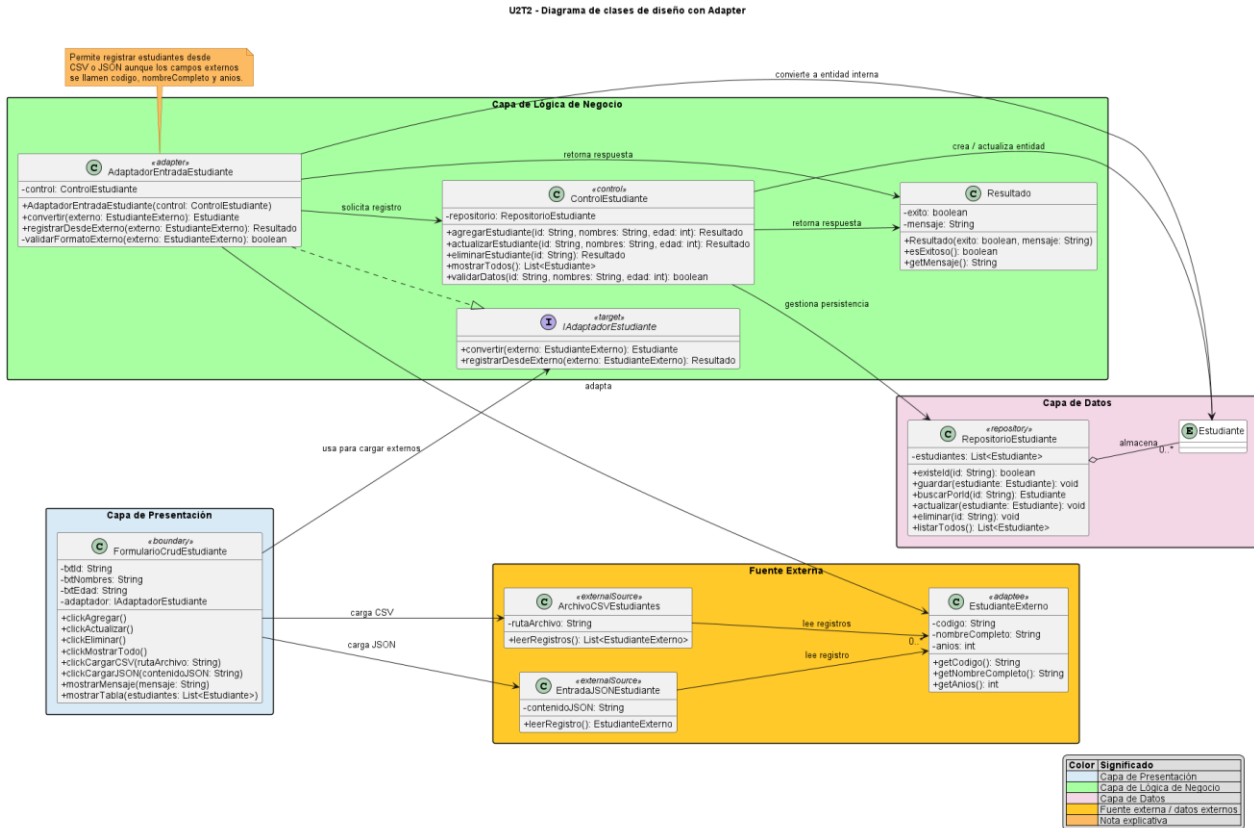
U2T2. Aplicación de Adapter y Decorator en el CRUD del estudiante

7. Definición del caso de información externa



De acuerdo con el modelo arquitectónico planteado, el sistema requiere cargar información de estudiantes proveniente de una Fuente Externa (como archivos CSV o JSON, representados por las clases ArchivoCSVEstudiantes y EntradaJSONEstudiante). Sin embargo, los datos provenientes de estos sistemas heredados o externos no son compatibles con el modelo interno del sistema. El formato externo genera un objeto EstudianteExterno que utiliza los atributos: código (en lugar de ID), nombreCompleto (en lugar de nombres) y años (en lugar de edad).

8. Diseño e implementación del Adapter

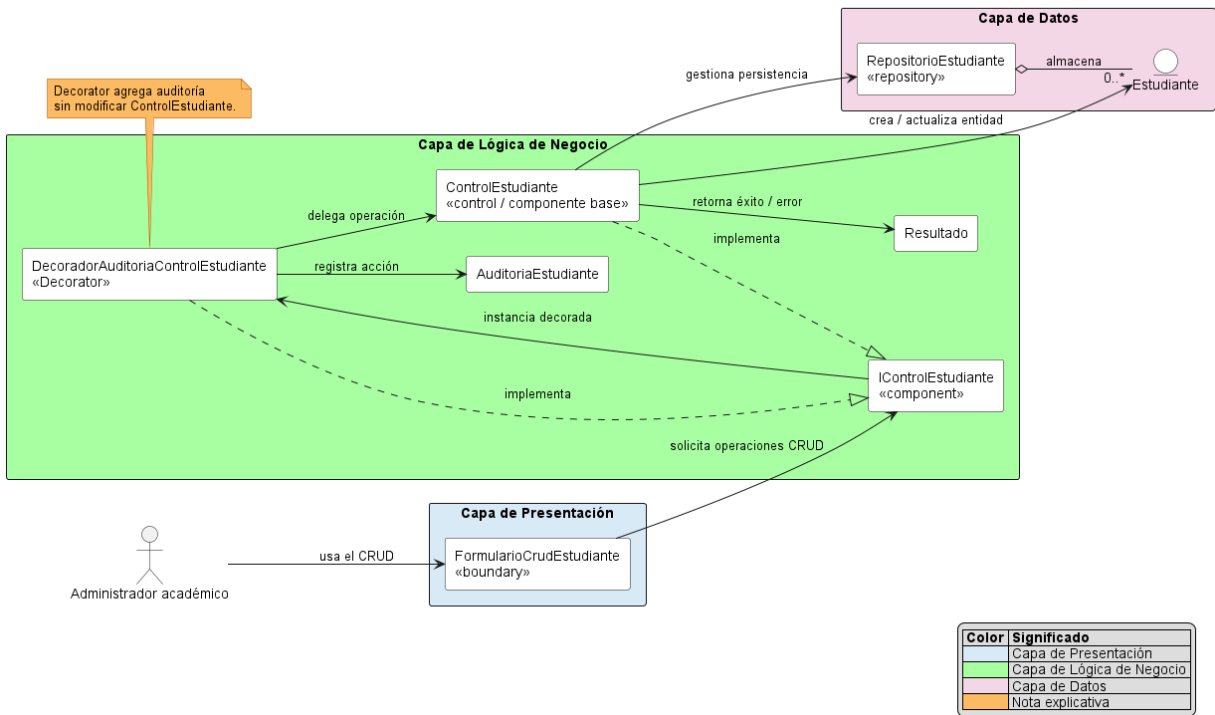


Para evitar modificar la lógica base de nuestro CRUD, se implementa el patrón Adapter.

- **Target:** Se define la interfaz `IAdaptadorEstudiante` con los métodos `convertir()` y `registrarDesdeExterno()`.
- **Adapter:** Se crea la clase `AdaptadorEntradaEstudiante`. Su responsabilidad es recibir al `EstudianteExterno`, extraer sus campos y mapearlos explícitamente a nuestra entidad principal `Estudiante` realizando la siguiente conversión: código -> id, nombreCompleto -> nombres, y años -> edad.
- **Integración:** Una vez adaptado el objeto, el Adapter se comunica con la capa de Lógica de Negocio (`ControlEstudiante`) para procesar el guardado utilizando las validaciones nativas del sistema.

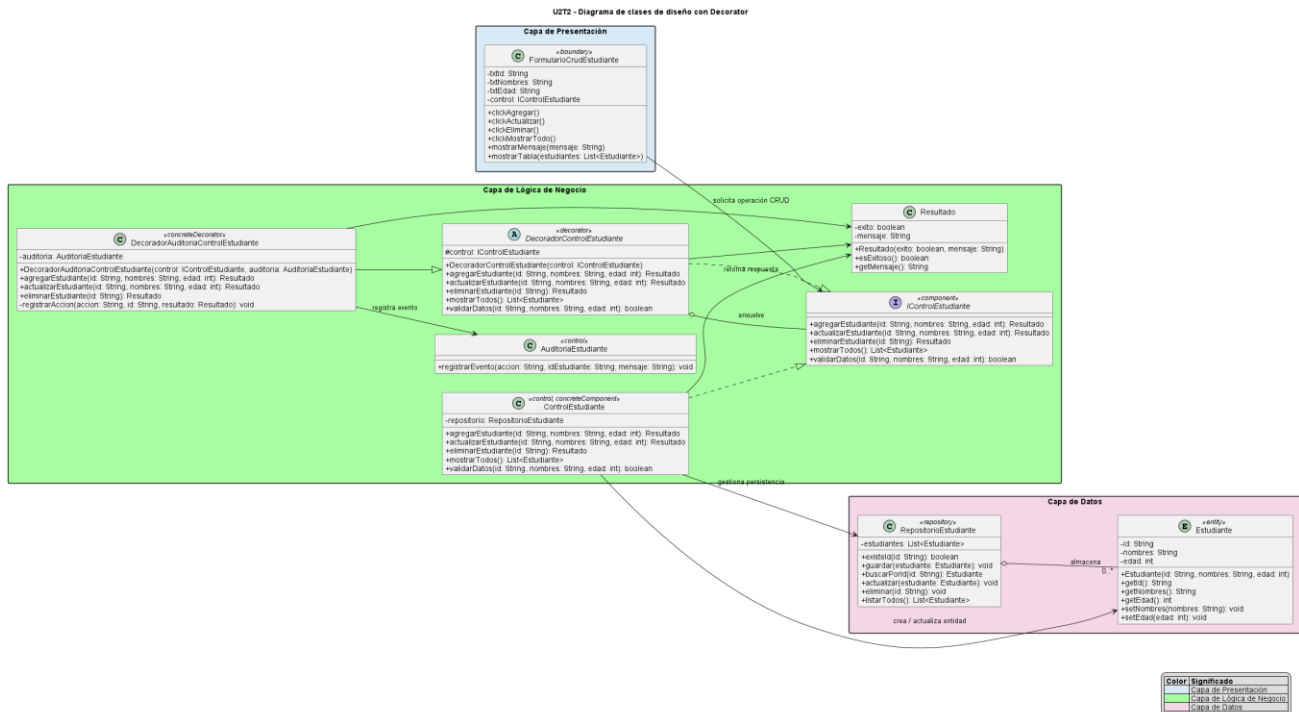
9. Definición de comportamiento adicional

U2T2 - Arquitectura del CRUD de Estudiantes con Decorators



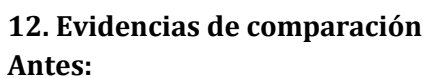
Se requiere incorporar una funcionalidad de Auditoría del Sistema. El objetivo es que cada vez que se ejecute una operación crítica en el CRUD (como agregar, actualizar o eliminar un estudiante), el sistema registre un log o bitácora de la acción. Este comportamiento debe añadirse sin modificar directamente el código fuente de la clase base del servicio (ControlEstudiante), cumpliendo con el principio Abierto/Cerrado (Open/Closed Principle).

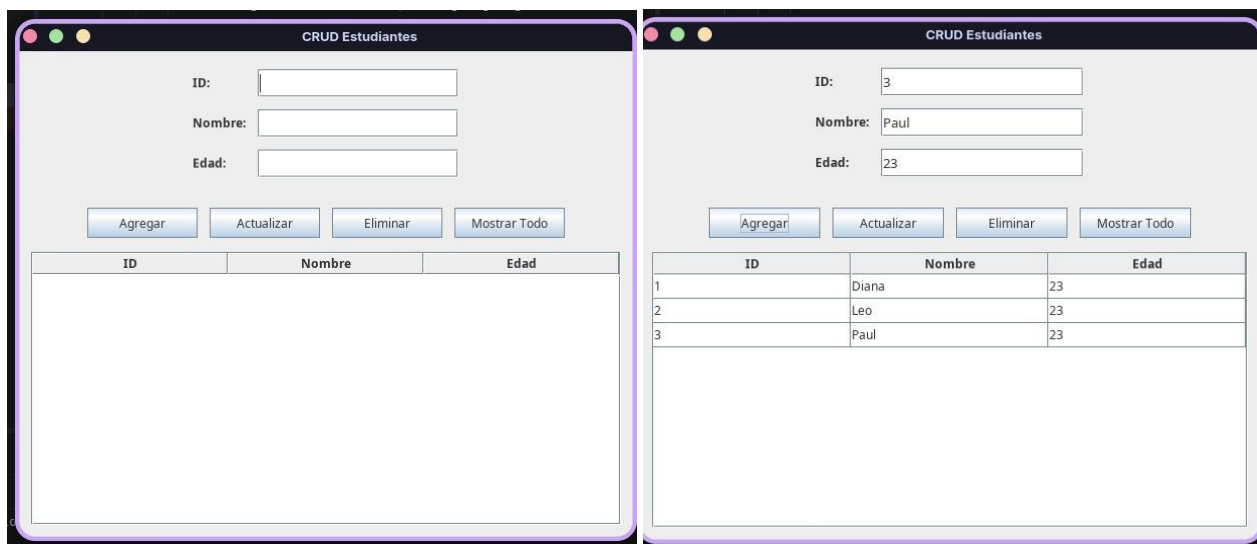
10. Diseño e implementación del Decorator



Para lograr esta extensión de funcionalidad, se implementa el patrón Decorator modificando ligeramente la estructura para basarla en abstracciones:

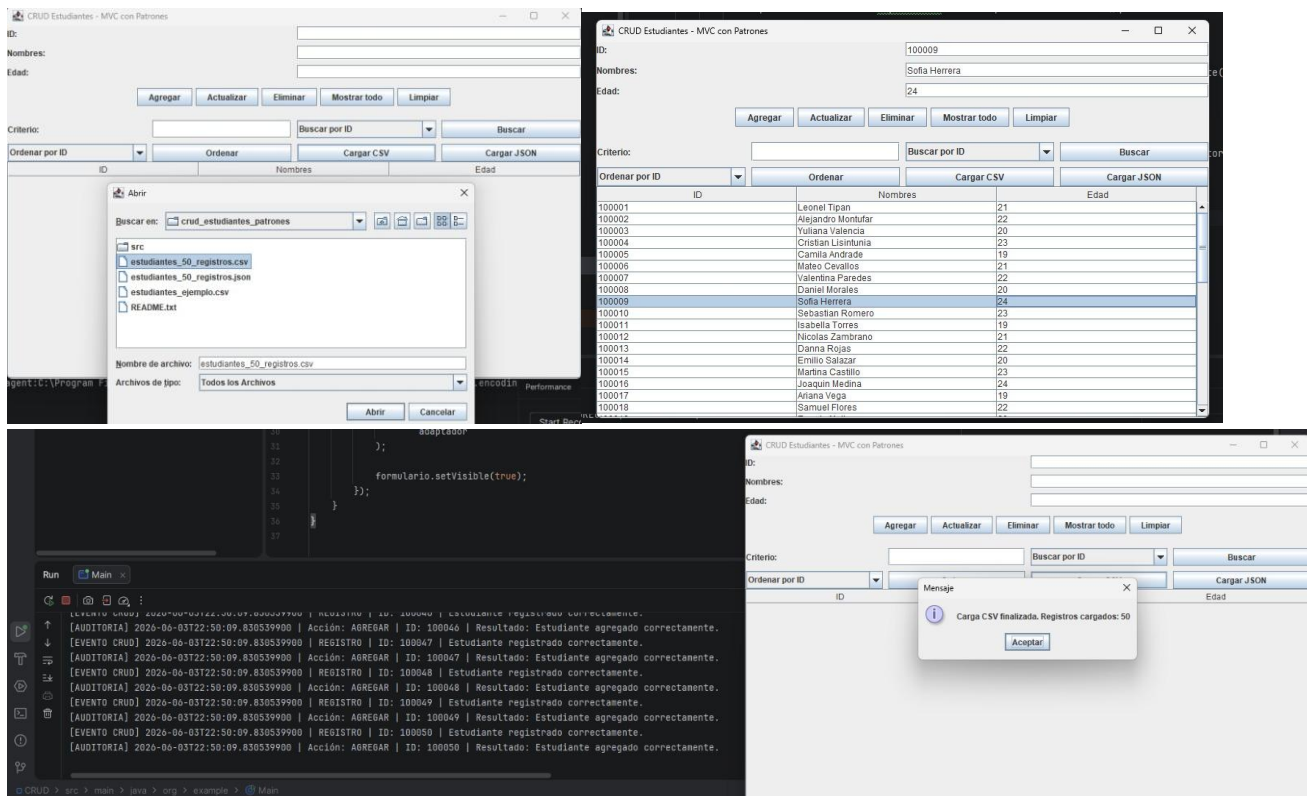
- ## 11. Actualización de los diagramas para evidenciar Adapter y Decorator





El sistema solo permitía ingresos manuales desde la interfaz y el controlador procesaba los datos silenciosamente. No existía rastro de qué usuario hizo qué operación, limitando el control administrativo, y no había forma de ingresar estudiantes desde archivos heredados.

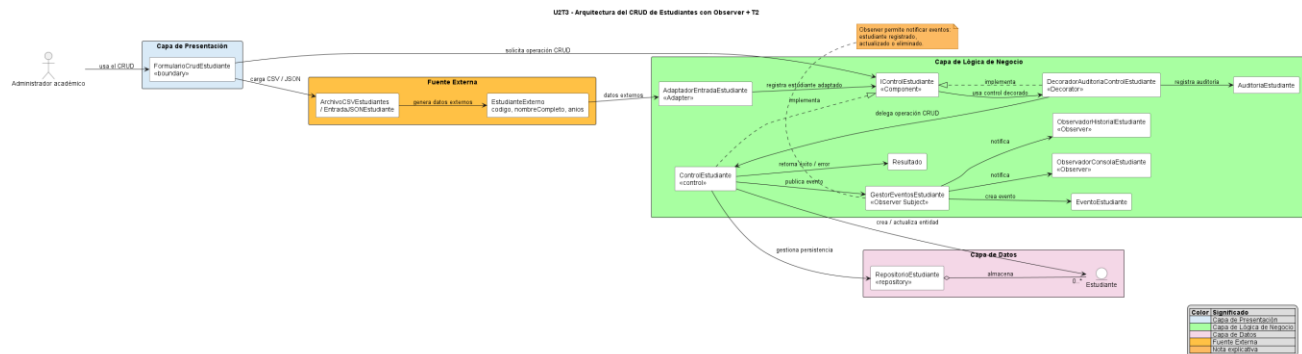
Después:



Prueba de Adapter: Al hacer clic en "Cargar CSV" o "Cargar JSON" desde la interfaz, el sistema lee los registros externos (codigo, nombreCompleto, anios), los traduce de forma transparente y los muestra exitosamente en la tabla como entidades estándar (ID, Nombres, Edad).

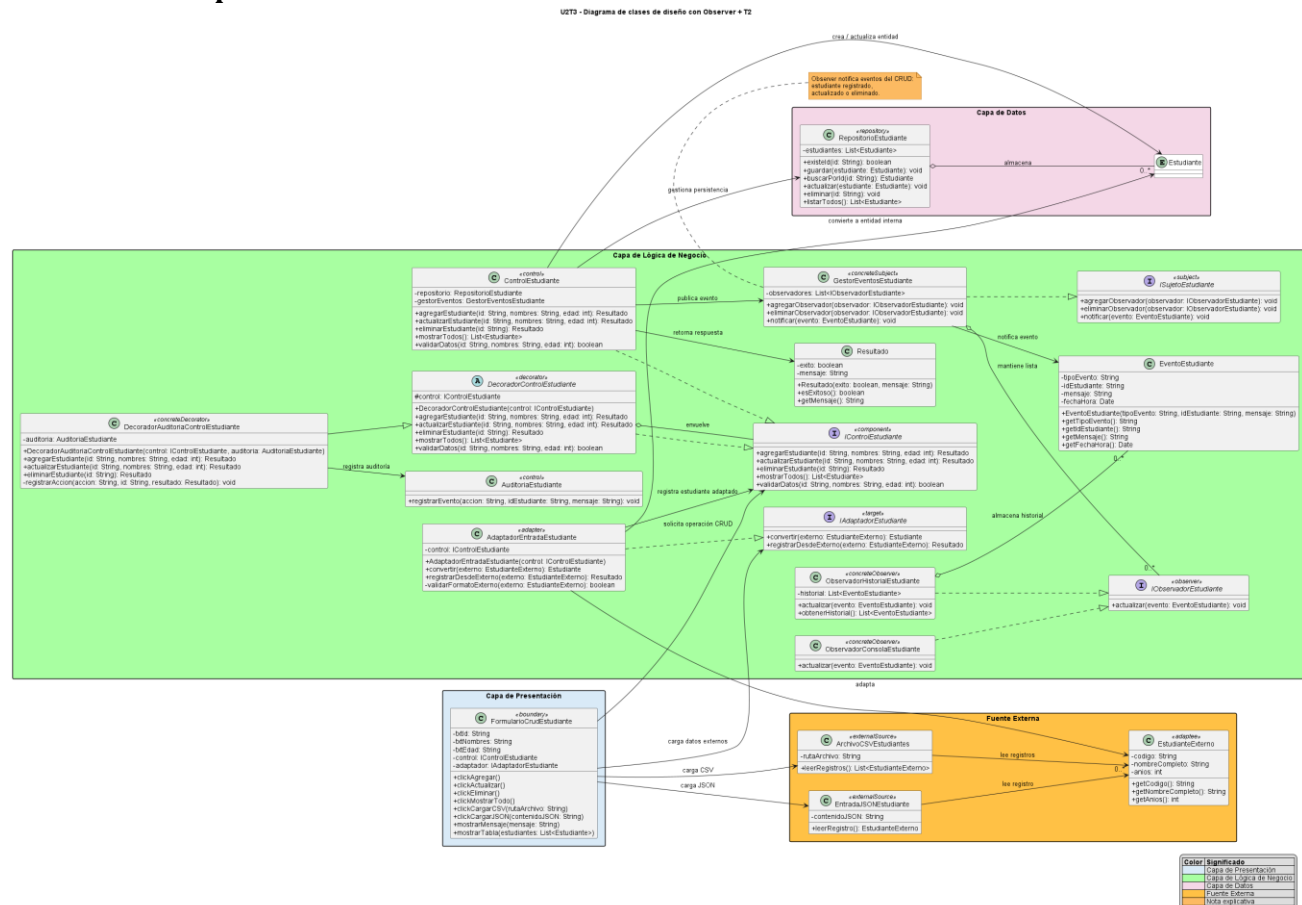
Prueba de Decorator: Al registrar o eliminar manualmente a un estudiante, la operación transcurre normal en pantalla, pero en consola, se imprime un registro generado por AuditoriaEstudiante, demostrando que el Decorator interceptó la operación exitosamente.

13. Identificación de un evento relevante del CRUD



Según el diseño arquitectónico establecido, el sistema requiere reaccionar ante los cambios de estado de los datos. Se han identificado como eventos relevantes las operaciones principales de modificación del CRUD: estudiante registrado, estudiante actualizado y estudiante eliminado. Estas acciones generan notificaciones críticas que otras partes del sistema necesitan conocer en tiempo real.

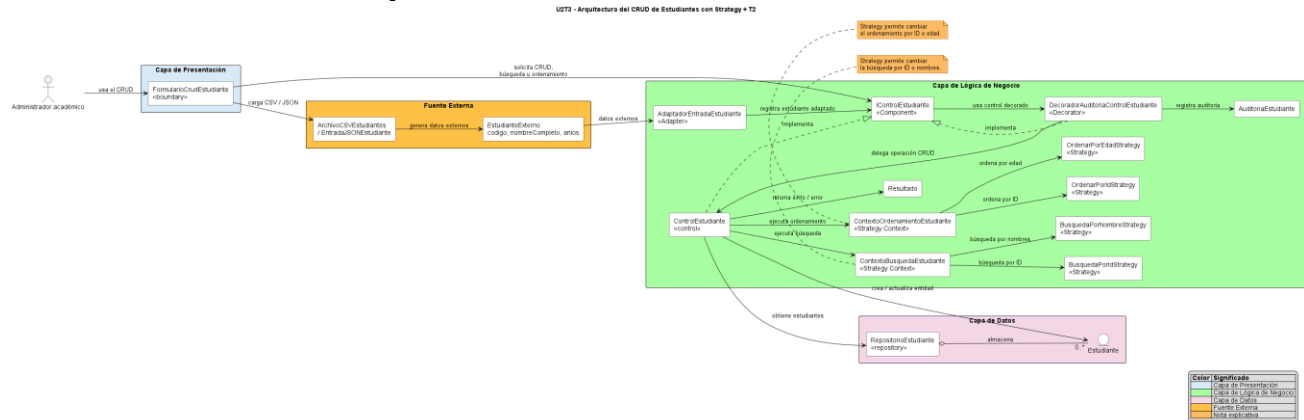
14. Diseño e implementación de Observer



Para gestionar estos eventos sin acoplar el controlador principal con los sistemas de registro, se implementó el patrón Observer de la siguiente manera:

- **Sujeto (Subject):** Se definió la interfaz ISujetoEstudiante y su implementación concreta GestorEventosEstudiante. Esta clase gestiona una lista de observadores y tiene la responsabilidad de notificarlos mediante el método notificar (evento: EventoEstudiante).
- **Observadores (Observers):** Se definió la interfaz IObservadorEstudiante. A partir de ella, se crearon dos observadores concretos:
 - ObservadorHistorialEstudiante: Mantiene una lista en memoria y almacena el historial de los eventos del sistema.
 - ObservadorConsolaEstudiante: Imprime los eventos directamente en la consola para propósitos de depuración o bitácora.
- **Evento:** Se encapsuló la información de la notificación en la clase EventoEstudiante, que contiene el tipo de evento, ID del estudiante, un mensaje descriptivo y la fecha/hora de ejecución.

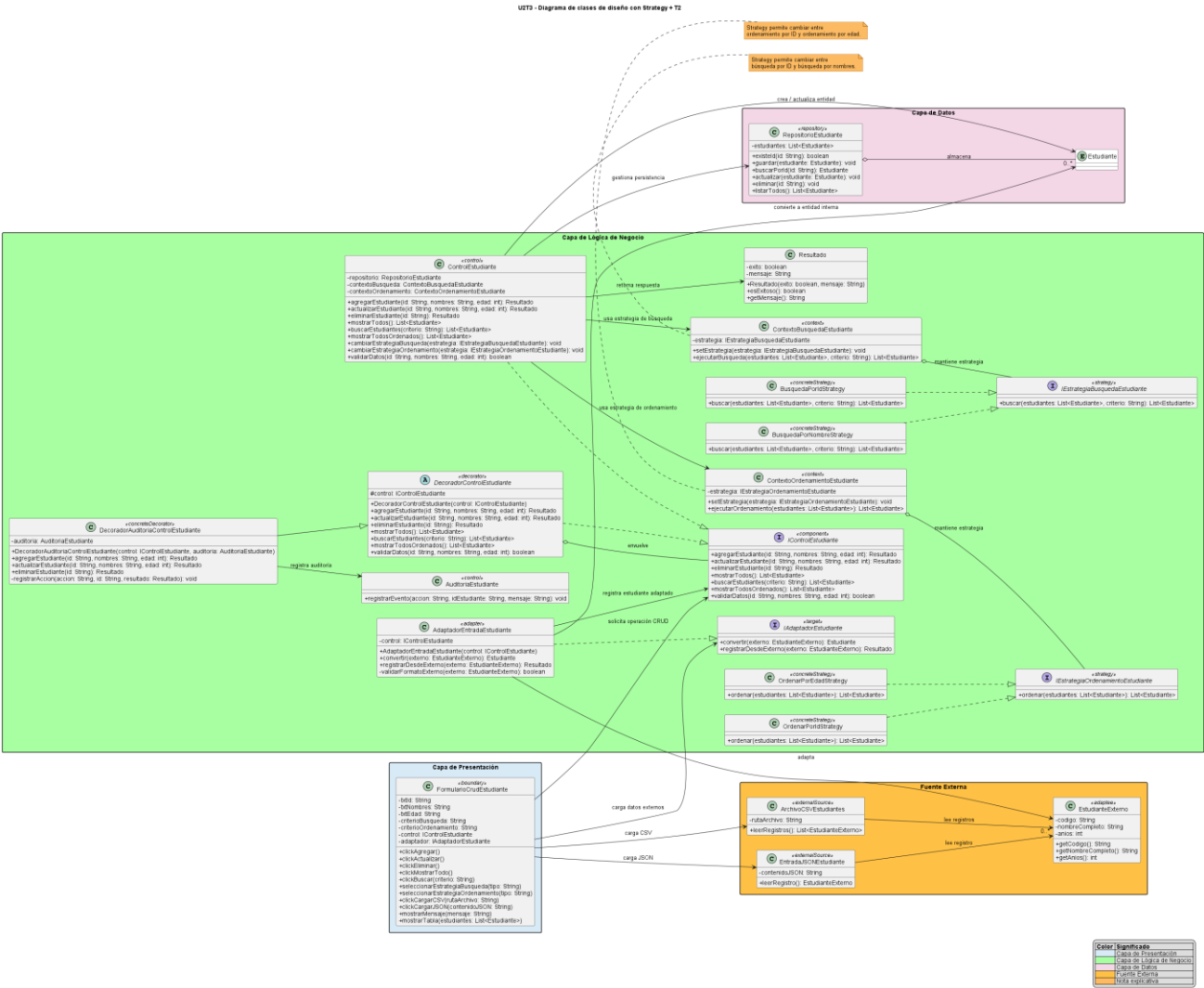
15. Identificación de un comportamiento variable



El sistema requiere flexibilidad para presentar y filtrar la información en la tabla de la interfaz. De acuerdo con el diseño, se identificaron dos comportamientos altamente variables:

- **Variación en la Búsqueda:** El usuario necesita poder cambiar dinámicamente entre buscar a un estudiante por su ID o buscarlo por sus Nombres.
- **Variación en el Ordenamiento:** El usuario necesita visualizar la tabla de estudiantes ordenada según diferentes criterios, específicamente ordenar por ID u ordenar por Edad.

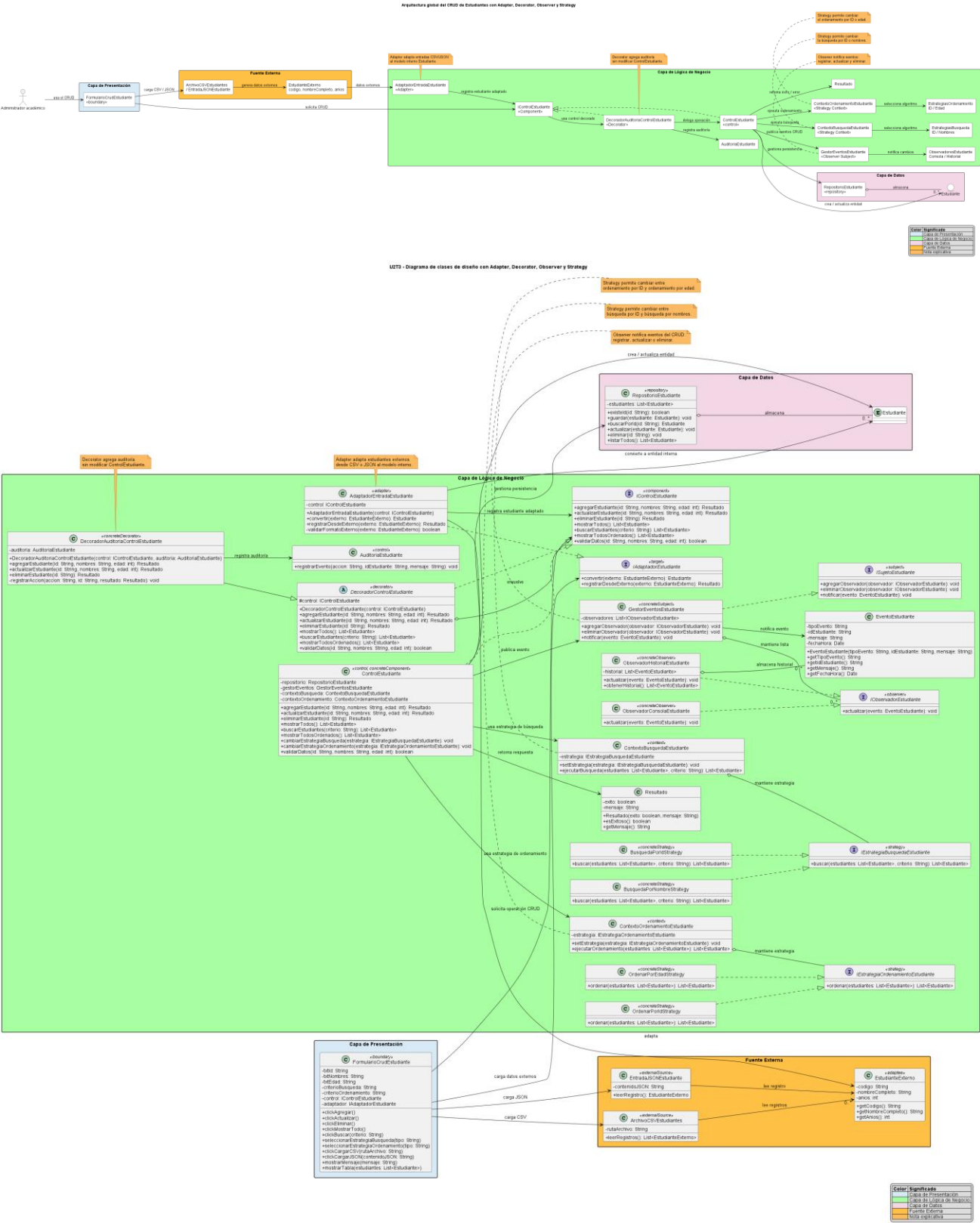
16. Diseño e implementación de Strategy



Para resolver las variaciones de comportamiento sin llenar el controlador de sentencias condicionales complejas, se aplicó el patrón Strategy utilizando interfaces comunes:

- **Estrategias de Búsqueda:** Se creó la interfaz `IEstrategiaBusquedaEstudiante`. A partir de ella se implementaron dos algoritmos concretos: `BusquedaPorIdStrategy` y `BusquedaPorNombreStrategy`. El control delega la acción a `ContextoBusquedaEstudiante`, el cual mantiene y ejecuta la estrategia seleccionada.
- **Estrategias de Ordenamiento:** Se creó la interfaz `IEstrategiaOrdenamientoEstudiante`. Se implementaron las estrategias concretas `OrdenarPorIdStrategy` y `OrdenarPorEdadStrategy`. La ejecución es administrada por `ContextoOrdenamientoEstudiante`.

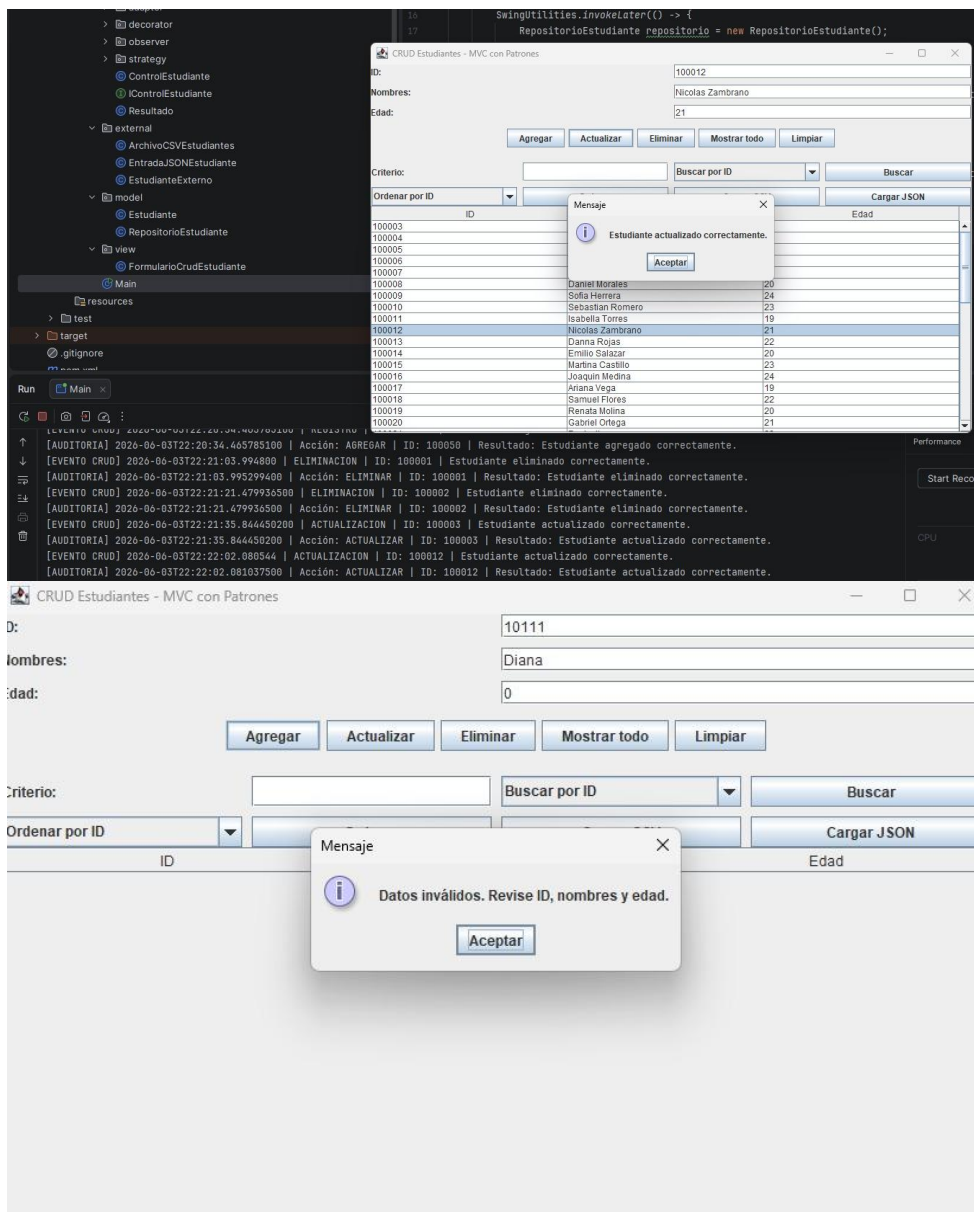
17. Actualización de la documentación del diseño



17. Matriz de Trazabilidad

RF	Caso de uso / Escenario	Patrón Aplicado	Evento / Estrategia / Extensión	Componentes Modificados
Escenario de Integración (Compatibilidad de formatos)	Cargar estudiantes desde fuente externa (CSV/JSON)	Adapter	Adaptación: Carga masiva de CSV y estandarización de datos (convierte código, nombreCompleto y años al modelo estándar de ID, Nombres y Edad).	Target: IAdaptadorEstudianteAdapter: AdaptadorEntradaEstudiante
RF-05 (Validar datos obligatorios y auditar)	Validar datos del estudiante / Registrar bitácora	Decorator	Extensión de comportamiento: Validación adicional de datos (ej. validar que la edad sea mayor a cero) y registro de auditoría al crear/actualizar, sin modificar el servicio base.	Componente: IControlEstudiante Decoradores: DecoradorControlEstudiante, DecoradorAuditoriaControlEstudiante
RF-01, RF-02, RF-03 (Agregar, Actualizar, Eliminar)	Agregar, Actualizar, Eliminar estudiante	Observer	Evento: Notificación automática cuando un estudiante es registrado, actualizado o eliminado.	Sujeto: GestorEventosEstudiante Observadores: ObservadorHistorialEstudiante, ObservadorConsolaEstudiante
RF-04 (Mostrar todos los estudiantes registrados)	Mostrar todos los estudiantes	Strategy	Comportamiento variable: Selección dinámica para buscar (por ID o Nombre) y ordenar (por ID o Edad) la tabla.	Contextos: ContextoBusquedaEstudiante, ContextoOrdenamientoEstudiante Estrategias: BusquedaPorIdStrategy, OrdenarPorEdadStrategy, etc.

18. Pruebas y evidencias de ejecución



CONCLUSIONES

- **U2T1:** La implementación de la arquitectura de tres capas ayuda a separar las responsabilidades del sistema CRUD. Se comprobó que aislar la capa de Presentación para que no acceda directamente a los Datos y centralizar las reglas en la Lógica de Negocio, genera un código más ordenado.
- **U2T2:** La integración de los patrones estructurales permitió aumentar las funcionalidades del sistema sin romper la arquitectura base. El patrón *Adapter* demostró ser útil para compatibilizar información externa con el modelo interno del estudiante, mientras que *Decorator* facilitó la adición de nuevos comportamientos sin modificar las clases originales.

- **U2T3:** La aplicación de patrones de comportamiento dio mayor flexibilidad y dinamismo. Mediante *Observer*, el sistema ahora puede reaccionar y notificar automáticamente eventos importantes, y gracias a *Strategy*, es posible intercambiar algoritmos en tiempo de ejecución, demostrando así la alta mantenibilidad y escalabilidad del software.

RECOMENDACIONES

- Se recomienda que ningún patrón de diseño se implemente sin estar justificado por un requisito o necesidad específica. Además, toda evidencia debe estar vinculada explícitamente a una tarea y a un requisito
- Se recomienda verificar constantemente las responsabilidades de las capas. Hay que asegurar que la capa de Presentación interactúe únicamente con la Lógica de Negocio y en ninguna circunstancia tenga responsabilidades de acceso a datos o reglas complejas
- Tener en cuenta que la aplicación de patrones de diseño debe enfocarse en hacer que el sistema sea más flexible y fácil de mantener a futuro, no en hacerlo más complejo. Cada patrón aplicado debe reducir el acoplamiento y facilitar el cambio de comportamientos

BIBLIOGRAFIA

- Adapter. (n.d.). <https://refactoring.guru/es/design-patterns/adapter>
- Decorator. (n.d.). <https://refactoring.guru/es/design-patterns/decorator>
- Observer. (n.d.). <https://refactoring.guru/es/design-patterns/observer>
- Strategy. (n.d.). <https://refactoring.guru/es/design-patterns/strategy>

ANEXOS